

Contrato API

Servicio de Despacho y Logística

Grupo 6

Especificación de Endpoints REST
Contratos JSON de Eventos
Matriz de Dependencias

Proyecto: Marketplace en la Nube
Versión API: v1.0
Base URL: <https://g6-despacho.onrender.com/api/v1>
Fecha: 11 de Junio de 2026
Estado: Borrador para revisión con G5 y G9

! Para revisión conjunta

Este documento debe ser validado con los equipos de **G5 (Pedidos)** y **G9 (Notificaciones)** antes de comenzar la implementación. Cualquier cambio acordado se reflejará en la versión 1.1.

Índice

I	Contrato REST API	3
1.	Información General	3
1.1.	Convenciones	3
1.2.	Resumen de Endpoints	3
2.	Modelo de Datos: Shipment	3
2.1.	Estados válidos	4
3.	Endpoints Detallados	4
3.1.	Crear Despacho	4
3.2.	Consultar Despacho por ID	5
3.3.	Buscar Despacho por Order ID	6
3.4.	Actualizar Estado del Despacho	6
3.5.	Listar Despachos	7
3.6.	Health Check	8
4.	Estructura Estándar de Errores	8
II	Contrato de Eventos JSON	9
5.	Mecanismo de Comunicación	9
5.1.	Flujo de Webhooks (propuesto)	9
6.	Formato Estándar del Evento	9
7.	Catálogo de Eventos	9
7.1.	ShipmentCreated	9
7.2.	ShipmentInTransit	10
7.3.	ShipmentDelivered	11
7.4.	ShipmentCancelled	11
7.5.	ShipmentFailed	11
7.6.	ShipmentReturned	12
III	Contratos Inter-Grupo	13
8.	Contrato con G5 (Pedidos) — Entrada	13
8.1.	Flujo de Integración	13
8.2.	JSON que G5 nos envía	13
8.3.	JSON que G6 le responde a G5	13
8.4.	Preguntas pendientes para G5	14
9.	Contrato con G9 (Notificaciones) — Salida	14
9.1.	Flujo de Integración	14
9.2.	JSON que G6 le envía a G9	14
9.3.	Respuesta esperada de G9	14
9.4.	Preguntas pendientes para G9	15

10. Contrato con G10 (Reportería) — Salida	15
10.1. Opciones de Integración	15
10.2. Datos relevantes para G10	15
11. Contrato con G1 (Frontend) — Consulta	15
 IV Matriz de Dependencias y Arquitectura	 16
12. Matriz de Dependencias	16
13. Diagrama de Arquitectura	16
14. Esquema SQL de la Base de Datos	16
15. Historial de Versiones	17

Parte I

Contrato REST API

1. Información General

Campo	Valor
Título	API de Despacho y Logística — Grupo 6
Versión	1.0.0
Base URL	<code>https://g6-despacho.onrender.com/api/v1</code>
Formato	JSON (<code>application/json</code>)
Autenticación	Ninguna (proyecto académico)
Encoding	UTF-8

1.1 Convenciones

- Todos los endpoints retornan `application/json`.
- Las fechas usan formato ISO 8601: `YYYY-MM-DDTHH:mm:ssZ`.
- Los IDs de shipment usan formato `SHP-<uuid>` (generados por nuestro servicio).
- Los errores siguen estructura estándar (ver Sección 4).

1.2 Resumen de Endpoints

Método	Ruta	Descripción	Consumidor
POST	<code>/shipments</code>	Crear un nuevo despacho	G5
GET	<code>/shipments/{id}</code>	Consultar estado de un despacho	G1, G5
GET	<code>/shipments?order_id={x}</code>	Buscar despacho por pedido	G5
PATCH	<code>/shipments/{id}</code>	Actualizar estado del despacho	Interno
GET	<code>/shipments</code>	Listar todos los despachos	G10
GET	<code>/health</code>	Health check del servicio	Todos

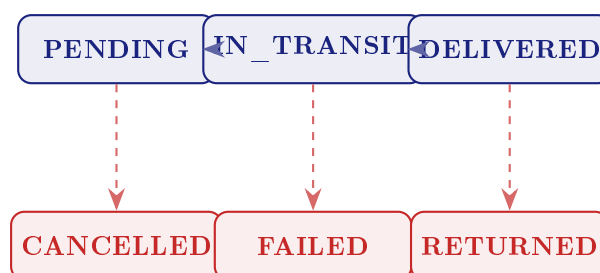
Cuadro 1: Resumen de endpoints expuestos por el Grupo 6.

2. Modelo de Datos: Shipment

Campo	Tipo	Ejemplo	Req.	Descripción
shipment_id	string	SHP-a1b2c3d4	—	ID único generado por G6.
order_id	string	ORD-20260611-001	Sí	ID del pedido (de G5).
customer_name	string	María G.	Sí	Nombre del destinatario.
address	string	Av. Prov. 1234	Sí	Dirección de entrega.
city	string	Santiago	Sí	Ciudad de destino.
weight_kg	number	3.2	Sí	Peso del paquete en kg.
status	string	PENDING	—	Estado actual.
created_at	datetime	2026-06-11T14:00Z	—	Fecha de creación.
updated_at	datetime	2026-06-11T15:30Z	—	Última actualización.
estimated_delivery	datetime	2026-06-14T18:00Z	—	Fecha estimada.

Cuadro 2: Esquema del recurso Shipment. “Req.” indica si es requerido en el POST.

2.1 Estados válidos



Estado	Valor	Tipo	Transiciones válidas
Pendiente	PENDING	Inicial	→ IN_TRANSIT, CANCELLED
En tránsito	IN_TRANSIT	Normal	→ DELIVERED, FAILED
Entregado	DELIVERED	Final	→ RETURNED
Cancelado	CANCELLED	Final	Ninguna (terminal).
Fallido	FAILED	Final	Ninguna (terminal).
Devuelto	RETURNED	Final	Ninguna (terminal).

Cuadro 3: Tabla de transiciones de estado.

3. Endpoints Detallados

3.1 Crear Despacho

POST /api/v1/shipments

Descripción: Crea una nueva orden de despacho. Llamado por **G5 (Pedidos)** cuando un pedido ha sido confirmado y pagado.

Request Body (application/json):

```
{
  "order_id": "ORD-20260611-001",
  "customer_name": "María González",
  "address": "Av. Providencia 1234, Depto 56",
  "city": "Santiago",
  "weight_kg": 3.2
}
```

Campos requeridos:

Campo	Tipo	Req.	Validaciones
order_id	string	Sí	No vacío. Único.
customer_name	string	Sí	No vacío. Máx. 200 chars.
address	string	Sí	No vacío. Máx. 500 chars.
city	string	Sí	No vacío. Máx. 100 chars.
weight_kg	number	Sí	Mayor a 0. Máx. 500 kg.

Response — 201 Created:

```
{
  "shipment_id": "SHP-a1b2c3d4",
  "order_id": "ORD-20260611-001",
  "customer_name": "María González",
  "address": "Av. Providencia 1234, Depto 56",
  "city": "Santiago",
  "weight_kg": 3.2,
  "status": "PENDING",
  "created_at": "2026-06-11T14:00:00Z",
  "updated_at": "2026-06-11T14:00:00Z",
  "estimated_delivery": "2026-06-14T18:00:00Z"
}
```

Errores:

Código	Nombre	Causa
400	Bad Request	JSON mal formado o campos faltantes.
409	Conflict	Ya existe despacho para ese order_id .
422	Unprocessable Entity	Datos inválidos (ej: peso negativo).

3.2 Consultar Despacho por ID

GET /api/v1/shipments/{shipment_id}

Descripción: Retorna estado actual y datos completos de un despacho. Llamado por **G1 (Frontend)** y **G5 (Pedidos)**.

Response — 200 OK:

```
{
  "shipment_id": "SHP-a1b2c3d4",
  "order_id": "ORD-20260611-001",
  "customer_name": "María González",
  "address": "Av. Providencia 1234, Depto 56",
  "city": "Santiago",
  "weight_kg": 3.2,
  "status": "IN_TRANSIT",
  "created_at": "2026-06-11T14:00:00Z",
  "updated_at": "2026-06-11T14:30:00Z",
  "estimated_delivery": "2026-06-14T18:00:00Z"
}
```

Errores:

Código	Nombre	Causa
404	Not Found	No existe despacho con ese ID.

3.3 Buscar Despacho por Order ID

GET /api/v1/shipments?order_id={order_id}

Descripción: Busca un despacho por el ID de pedido de G5.

Response — 200 OK:

```
{
  "shipment_id": "SHP-a1b2c3d4",
  "order_id": "ORD-20260611-001",
  "status": "PENDING",
  "created_at": "2026-06-11T14:00:00Z",
  "estimated_delivery": "2026-06-14T18:00:00Z"
}
```

Errores:

Código	Nombre	Causa
400	Bad Request	Falta parámetro <code>order_id</code> .
404	Not Found	No existe despacho para ese pedido.

3.4 Actualizar Estado del Despacho

PATCH /api/v1/shipments/{shipment_id}

Descripción: Actualiza el estado de un despacho. Uso **interno** (simulación automática) o por **G5** para cancelar.

Request Body:

```
{
  "status": "IN_TRANSIT"
}
```

Response — 200 OK:

```
{
  "shipment_id": "SHP-a1b2c3d4",
  "order_id": "ORD-20260611-001",
  "status": "IN_TRANSIT",
  "previous_status": "PENDING",
  "updated_at": "2026-06-11T14:30:00Z"
}
```

Nota

Al cambiar el estado exitosamente, el servicio emite automáticamente el evento correspondiente hacia G9 (ver Parte II).

Errores:

Código	Nombre	Causa
400	Bad Request	Estado no es un valor válido.
404	Not Found	No existe despacho con ese ID.
409	Conflict	Transición no permitida (ej: DELIVERED → PENDING).

3.5 Listar Despachos

GET /api/v1/shipments

Descripción: Lista de despachos con filtros. Para **G10 (Reportería)**.

Query Parameters (opcionales):

Param	Tipo	Default	Descripción
status	string	(todos)	Filtrar por estado.
limit	int	50	Máx. resultados.
offset	int	0	Paginación.

Response — 200 OK:

```
{
  "total": 142,
  "limit": 50,
  "offset": 0,
  "shipments": [
```



```
{
  "shipment_id": "SHP-a1b2c3d4",
  "order_id": "ORD-20260611-001",
  "status": "DELIVERED",
  "city": "Santiago",
  "created_at": "2026-06-11T14:00:00Z",
  "updated_at": "2026-06-11T16:00:00Z"
}
```

3.6 Health Check

GET /api/v1/health

```
{
  "status": "ok",
  "service": "g6-despacho",
  "version": "1.0.0",
  "timestamp": "2026-06-11T14:00:00Z"
}
```

4. Estructura Estándar de Errores

Todos los errores siguen el mismo formato:

```
{
  "error": {
    "code": 422,
    "type": "VALIDATION_ERROR",
    "message": "El campo weight_kg debe ser mayor a 0.",
    "details": {
      "field": "weight_kg",
      "value": -1.5,
      "constraint": "must be greater than 0"
    }
  }
}
```

Código	type	Uso
400	BAD_REQUEST	JSON malformado, campos faltantes.
404	NOT_FOUND	Recurso no encontrado.
409	CONFLICT	Duplicado o transición inválida.
422	VALIDATION_ERROR	Datos que no pasan validación.
500	INTERNAL_ERROR	Error inesperado del servidor.

Cuadro 4: Códigos de error estandarizados.

Parte II

Contrato de Eventos JSON

5. Mecanismo de Comunicación

! Decisión pendiente con G9

Se propone usar **Webhooks HTTP**: nuestro servicio hace un **POST** a una URL de G9 cada vez que cambia un estado.

Alternativa: G9 hace polling a nuestro **GET**.

Acordar en la reunión de contratos antes del 16 de Junio.

5.1 Flujo de Webhooks (propuesto)

1. G9 nos registra su URL de webhook.
2. Cada cambio de estado dispara un **POST** a esa URL.
3. G9 responde **200 OK** para confirmar recepción.
4. Si falla, reintentamos 3 veces con backoff (1s, 2s, 4s).

6. Formato Estándar del Evento

Todos los eventos emitidos por G6 siguen esta estructura:

📁 Envelope del Evento (estructura común)

```
{
  "event_id": "evt-550e8400-e29b-41d4-a716-446655440000",
  "event_type": "ShipmentDelivered",
  "source": "g6-despacho",
  "timestamp": "2026-06-11T15:00:00Z",
  "payload": { ... }
}
```

7. Catálogo de Eventos

7.1 ShipmentCreated

Se emite cuando G5 crea exitosamente un nuevo despacho.

Campo	Tipo	Descripción
event_id	string (UUID)	Identificador único del evento.
event_type	string	Tipo del evento (ver catálogo).
source	string	Siempre "g6-despacho".
timestamp	datetime	Momento del evento (ISO 8601).
payload	object	Datos específicos.

Cuadro 5: Campos del envelope de evento.

ShipmentCreated

```
{
  "event_id": "evt-550e8400-e29b-41d4-a716-446655440001",
  "event_type": "ShipmentCreated",
  "source": "g6-despacho",
  "timestamp": "2026-06-11T14:00:00Z",
  "payload": {
    "shipment_id": "SHP-a1b2c3d4",
    "order_id": "ORD-20260611-001",
    "customer_name": "María González",
    "address": "Av. Providencia 1234, Depto 56",
    "city": "Santiago",
    "weight_kg": 3.2,
    "status": "PENDING",
    "estimated_delivery": "2026-06-14T18:00:00Z"
  }
}
```

Consumidores: G9 (notifica al cliente), G10 (registro).

7.2 ShipmentInTransit

Se emite cuando el paquete sale a reparto.

ShipmentInTransit

```
{
  "event_id": "evt-550e8400-e29b-41d4-a716-446655440002",
  "event_type": "ShipmentInTransit",
  "source": "g6-despacho",
  "timestamp": "2026-06-11T14:30:00Z",
  "payload": {
    "shipment_id": "SHP-a1b2c3d4",
    "order_id": "ORD-20260611-001",
    "previous_status": "PENDING",
    "new_status": "IN_TRANSIT",
    "customer_name": "María González",
    "city": "Santiago"
  }
}
```

Consumidores: G9 (“Tu paquete va en camino”).

7.3 ShipmentDelivered

Se emite cuando el paquete es entregado exitosamente.

ShipmentDelivered

```
{
  "event_id": "evt-550e8400-e29b-41d4-a716-446655440003",
  "event_type": "ShipmentDelivered",
  "source": "g6-despacho",
  "timestamp": "2026-06-11T15:00:00Z",
  "payload": {
    "shipment_id": "SHP-a1b2c3d4",
    "order_id": "ORD-20260611-001",
    "previous_status": "IN_TRANSIT",
    "new_status": "DELIVERED",
    "customer_name": "María González",
    "city": "Santiago",
    "delivered_at": "2026-06-11T15:00:00Z"
  }
}
```

Consumidores: G9 (“Tu paquete ha llegado”), G10 (métricas).

7.4 ShipmentCancelled

Se emite cuando un despacho es cancelado antes de salir a reparto.

ShipmentCancelled

```
{
  "event_id": "evt-550e8400-e29b-41d4-a716-446655440004",
  "event_type": "ShipmentCancelled",
  "source": "g6-despacho",
  "timestamp": "2026-06-11T14:10:00Z",
  "payload": {
    "shipment_id": "SHP-a1b2c3d4",
    "order_id": "ORD-20260611-001",
    "previous_status": "PENDING",
    "new_status": "CANCELLED",
    "customer_name": "María González",
    "reason": "Pedido anulado por el cliente"
  }
}
```

Consumidores: G9 (notifica cancelación), G5 (confirma).

7.5 ShipmentFailed

Se emite cuando un intento de entrega falla.

ShipmentFailed

```
{
  "event_id": "evt-550e8400-e29b-41d4-a716-446655440005",
  "event_type": "ShipmentFailed",
  "source": "g6-despacho",
  "timestamp": "2026-06-11T16:00:00Z",
  "payload": {
    "shipment_id": "SHP-a1b2c3d4",
    "order_id": "ORD-20260611-001",
    "previous_status": "IN_TRANSIT",
    "new_status": "FAILED",
    "customer_name": "María González",
    "reason": "Destinatario ausente"
  }
}
```

Consumidores: G9 (“Hubo un problema con tu entrega”), G5.

7.6 ShipmentReturned

Se emite cuando un paquete entregado es devuelto.



ShipmentReturned

```
{
  "event_id": "evt-550e8400-e29b-41d4-a716-446655440006",
  "event_type": "ShipmentReturned",
  "source": "g6-despacho",
  "timestamp": "2026-06-12T10:00:00Z",
  "payload": {
    "shipment_id": "SHP-a1b2c3d4",
    "order_id": "ORD-20260611-001",
    "previous_status": "DELIVERED",
    "new_status": "RETURNED",
    "customer_name": "María González",
    "reason": "Producto incorrecto"
  }
}
```

Consumidores: G9, G5 (gestionar reembolso), G10 (métricas).

Parte III

Contratos Inter-Grupo

8. Contrato con G5 (Pedidos) — Entrada

Este contrato define **qué datos nos envía G5** cuando un pedido es confirmado y necesita ser despachado.

8.1 Flujo de Integración

1. El cliente completa el pago en G5.
2. G5 llama a nuestro `POST /api/v1/shipments`.
3. Respondemos con el `shipment_id` y estado `PENDING`.
4. G5 almacena el `shipment_id` para consultas futuras.

8.2 JSON que G5 nos envía

Request de G5 hacia G6

```
{
  "order_id": "ORD-20260611-001",
  "customer_name": "María González",
  "address": "Av. Providencia 1234, Depto 56",
  "city": "Santiago",
  "weight_kg": 3.2
}
```

8.3 JSON que G6 le responde a G5

Response de G6 hacia G5

```
{
  "shipment_id": "SHP-a1b2c3d4",
  "order_id": "ORD-20260611-001",
  "customer_name": "María González",
  "address": "Av. Providencia 1234, Depto 56",
  "city": "Santiago",
  "weight_kg": 3.2,
  "status": "PENDING",
  "created_at": "2026-06-11T14:00:00Z",
  "updated_at": "2026-06-11T14:00:00Z",
  "estimated_delivery": "2026-06-14T18:00:00Z"
}
```

8.4 Preguntas pendientes para G5

! Confirmar con G5

1. Formato de `order_id`: propuesto como string libre.
2. ¿Necesitan campos adicionales en la respuesta?
3. ¿Nos envían el peso o lo calculamos nosotros?
4. ¿Cancelan vía `PATCH` o necesitan endpoint dedicado?

9. Contrato con G9 (Notificaciones) — Salida

Define **qué datos le enviamos a G9** en cada cambio de estado.

9.1 Flujo de Integración

1. Un shipment cambia de estado (vía `PATCH` o simulación).
2. Construimos el evento JSON correspondiente.
3. Hacemos `POST` al webhook de G9.
4. G9 procesa y notifica al usuario final.

9.2 JSON que G6 le envía a G9

Evento de G6 hacia G9 (ejemplo: entrega)

```
{
  "event_id": "evt-550e8400-e29b-41d4-a716-446655440003",
  "event_type": "ShipmentDelivered",
  "source": "g6-despacho",
  "timestamp": "2026-06-11T15:00:00Z",
  "payload": {
    "shipment_id": "SHP-a1b2c3d4",
    "order_id": "ORD-20260611-001",
    "previous_status": "IN_TRANSIT",
    "new_status": "DELIVERED",
    "customer_name": "María González",
    "city": "Santiago",
    "delivered_at": "2026-06-11T15:00:00Z"
  }
}
```

9.3 Respuesta esperada de G9

Acknowledgment de G9

```
{
  "received": true,
  "event_id": "evt-550e8400-e29b-41d4-a716-446655440003"
}
```

9.4 Preguntas pendientes para G9

! Confirmar con G9

1. ¿URL de webhook?
2. ¿Campos adicionales necesarios? (email, teléfono)
3. ¿Prefieren webhooks o polling?
4. ¿Cómo confirman recepción? Proponemos 200 OK.
5. ¿Qué eventos les interesan? ¿Los 6 o solo algunos?

10. Contrato con G10 (Reportería) — Salida

G10 consume datos para dashboards de rendimiento.

10.1 Opciones de Integración

1. **Vía API REST:** GET /shipments con filtros.
2. **Vía Eventos:** Suscripción a webhooks (mismos que G9).

10.2 Datos relevantes para G10

Métrica	Cómo la obtienen
Total despachos	GET /shipments → total.
Por estado	Filtrar con ?status=DELIVERED.
Tasa de éxito	DELIVERED / total.
Tasa de fallos	(FAILED + RETURNED) / total.
Tiempo de entrega	updated_at - created_at en DELIVERED.

11. Contrato con G1 (Frontend) — Consulta

G1 consume nuestro endpoint de tracking.

Endpoint: GET /api/v1/shipments/{shipment_id}

Campo	Uso en la UI
status	Paso actual en la barra de progreso.
estimated_delivery	“Llega el 14 de Junio”.
city	“Tu paquete va camino a Santiago”.
updated_at	“Última actualización: hace 30 min”.

Parte IV

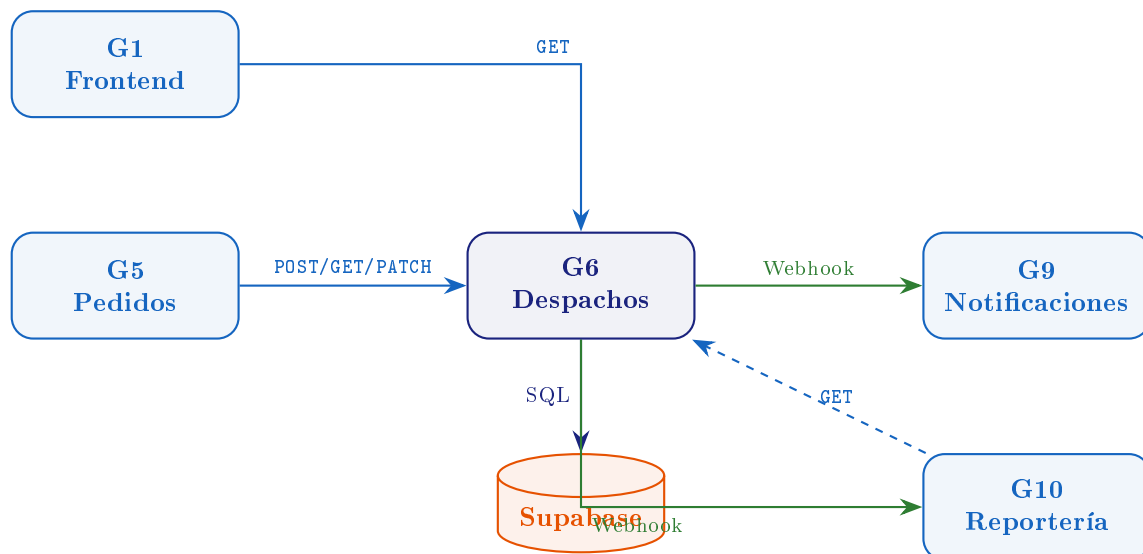
Matriz de Dependencias y Arquitectura

12. Matriz de Dependencias

Origen	Destino	Tipo	Protocolo	Datos
G5	G6	Síncrono	REST POST	Datos del pedido.
G5	G6	Síncrono	REST GET	Consulta por <code>order_id</code> .
G5	G6	Síncrono	REST PATCH	Cancelación.
G1	G6	Síncrono	REST GET	Tracking por <code>shipment_id</code> .
G6	G9	Asíncrono	Webhook POST	Eventos de estado.
G6	G10	Asíncrono	Webhook POST	Eventos de estado.
G10	G6	Síncrono	REST GET	Lista para reportes.

Cuadro 6: Matriz completa de dependencias del Grupo 6.

13. Diagrama de Arquitectura



14. Esquema SQL de la Base de Datos

```

CREATE TABLE shipments (
  shipment_id TEXT PRIMARY KEY,
  order_id TEXT UNIQUE NOT NULL,
  customer_name TEXT NOT NULL,
  address TEXT NOT NULL,
  city TEXT NOT NULL,
  weight_kg REAL NOT NULL CHECK (weight_kg > 0),
  status TEXT NOT NULL DEFAULT 'PENDING'
);

```

```
        CHECK (status IN (
                'PENDING', 'IN_TRANSIT', 'DELIVERED',
                'CANCELLED', 'FAILED', 'RETURNED'
        )),
    created_at      TIMESTAMP NOT NULL DEFAULT NOW(),
    updated_at      TIMESTAMP NOT NULL DEFAULT NOW(),
    estimated_delivery TIMESTAMP
);
```

i Nota

Esquema para Supabase (PostgreSQL). Si se usa Firebase, se adaptará a documentos JSON con los mismos campos.

15. Historial de Versiones

Ver.	Fecha	Autor	Cambios
1.0	11 Jun 2026	Grupo 6	Versión inicial.